

Laboratorio 2

Contents

1	Introduzione	3
2	Il terminale UNIX	4
2.1	Comandi	4
2.1.1	Struttura base comandi Linux	4
2.1.2	-help e man	4
2.1.3	pwd	4
2.1.4	ls	4
2.1.5	cd	5
2.1.6	mkdir	5
3	Linguaggio C	6
3.1	Compilatore	6
3.1.1	Warnings	6
3.2	Librerie	6
3.3	printf	6
3.3.1	Placeholders	6
3.4	Variabili	6
3.5	Commenti	7

1 Introduzione

In questo corso lavoreremo con programmi vicini al linguaggio macchina.
Lavoreremo inoltre con:

- UNIX
- thread
- processi
- programmazione di sistema

2 Il terminale UNIX

Il **terminale** è una interfaccia testuale per dialogare con il sistema. Dentro il terminale gira una **shell**, ovvero il programma che interpreta i comandi. Il terminale mostra il **prompt**, riceve i comandi e visualizza l'output.

2.1 Comandi

2.1.1 Struttura base comandi Linux

Esiste una struttura base per molti comandi, ovvero questa:

comando + opzione + argomento + (opzione + argomento)

per esempio:

```
$ gcc -Wall hello.c -o hello
```

2.1.2 -help e man

```
$ gcc --help
$ man gcc
```

Entrambi questi comandi aiutano a comprendere specifici comandi in modi diversi, `-help` è un aiuto generico mentre `"man"` è il manuale.

2.1.3 pwd

```
$ pwd
```

Il simbolo `$` indica che la shell è pronta a ricevere un comando (potrebbe non essere `$` a volte).

pwd è il comando "print working directory" e mostra la posizione della directory corrente all'interno del **file system**.

Il file system inizia dalla **root** ("`/`") e si articola nella directory, formando un **path**, ovvero un percorso attraverso directory, dove ogni passaggio è separato da "`/`" (potrebbe variare in altri ambienti).

Esistono due tipi di path:

- **Assoluto:** parte dalla root.
- **Relativo:** parte dalla directory corrente.

2.1.4 ls

```
$ ls
```

ls elenca tutto il contenuto della directory corrente, directory e file.

Può essere usata con alcuni **attributi**, aggiungibili con un "dash" (`-`) come in questo caso:

```
$ ls -a
```

Ecco gli attributi:

- **a:** include tutti gli elementi nascosti
- **l:** include informazioni avanzate (data, grandezza...)

2.1.5 cd

```
$ cd esercizi
$ cd /users/gabri/esercizi
```

cd significa "change directory" e si sposta nella directory specificata da un path relativo o assoluto.

Percorsi speciali:

- **Directory corrente:** specificato da un punto, ha lo stesso valore di `pwd`.

```
$ cd .
```

- **Directory padre:** specificato da un doppio punto.

```
$ cd ..
```

2.1.6 mkdir

```
$ mkdir esercizi
```

mkdir significa "make directory" e crea una directory del nome specificato nella cartella corrente.

3 Linguaggio C

Il linguaggio **C** è un linguaggio compilato, pertanto, prima di eseguire un file bisogna convertirlo in binario.

Esistono diversi standard C: C89, C95, C99, C11, C17, C23.

3.1 Compilatore

Il compilatore più conosciuto e usato per C è **gcc**, ovvero "GNU C Compiler".

```
gcc hello.c -o hello
```

In questo caso, il comando:

- compila il file "hello.c"
- rinomina l'eseguibile come "hello"

Ogni programma eseguito ritorna un risultato per determinare se è terminato con successo o meno, si ottiene così:

```
$ gcc hello.c
$ ./hello.c
$ echo $?
0
```

3.1.1 Warnings

I **warnings** sono importanti e non vanno ignorati, per compilarli e stamparli tutti si fa così:

```
$ gcc -Wall hello.c
```

Ogni warning contiene un messaggio e la location (riga e carattere) di esso.

3.2 Librerie

Nei programmi in C è possibile utilizzare delle **direttive di preprocessazione** (o del preprocessore) che permettono di importare funzioni e strumenti utili allo svolgimento del programma:

```
#include <stdio.h>
```

3.3 printf

3.3.1 Placeholders

3.4 Variabili

In C possiamo usare le **variabili**. Si possono inizializzare ad un valore di un tipo specifico, assegnato al momento della **dichiarazione**.

```
int x;
x = 32
```

3.5 Commenti

I commenti in C possono essere inseriti in vari modi:

- Commenti su una riga: aperti da `"/`, non vengono chiusi
- Commenti su più righe: delimitati da `/*` e da `*/`.